Bloque 4. Planificación y ejecución automática de tareas

La administración de un sistema gestor de bases de datos incluye numerosas operaciones que deben realizarse de forma periódica: copias de seguridad, mantenimiento, limpieza de información, comprobaciones de integridad y generación de informes.

Para evitar tener que ejecutar manualmente estas operaciones, podemos utilizar herramientas de planificación que permitan programar su ejecución en determinados momentos o intervalos.

> **Objetivos del bloque**
>
> - Comprender la importancia de planificar tareas administrativas.
> - Conocer el planificador de eventos integrado en MariaDB.
> - Crear eventos programados mediante `CREATE EVENT`.
> - Utilizar `cron` en Debian para ejecutar scripts SQL.
> - Automatizar copias de seguridad con `mariadb-dump`.
> - Registrar resultados y gestionar errores en tareas automatizadas.

1. Planificación de tareas administrativas

La planificación de tareas consiste en establecer cuándo y con qué frecuencia deben ejecutarse determinadas operaciones de administración. Una tarea puede ejecutarse una única vez, repetirse periódicamente o activarse en función de un calendario.

1.1. ¿Qué tareas se pueden automatizar?

Tarea	Descripción	Periodicidad habitual
Copias de seguridad	Generar copias de las bases de datos.	Diaria o semanal.
Limpieza de registros	Eliminar información temporal o caducada.	Diaria o mensual.
Generación de informes	Obtener estadísticas o resúmenes.	Diaria, semanal o mensual.
Comprobaciones	Verificar condiciones de integridad o disponibilidad.	Periódica.
Mantenimiento	Ejecutar operaciones de mantenimiento de las tablas.	Según necesidades.
Archivado	Trasladar información antigua a tablas históricas.	Mensual o anual.

1.2. Ventajas de la planificación

- **Reducción de tareas manuales:** evita que el administrador tenga que iniciar cada operación.
- **Regularidad:** las tareas se ejecutan siguiendo una planificación establecida.
- **Menor riesgo de olvidos:** las operaciones periódicas no dependen de que una persona las recuerde.
- **Mejor organización:** permite distribuir las operaciones en horarios de menor actividad.
- **Trazabilidad:** facilita registrar cuándo se ejecutó una tarea y cuál fue su resultado.

> **Importante**
> Automatizar una tarea no significa olvidarse de ella. Es necesario comprobar periódicamente que se ejecuta correctamente, que los resultados son válidos y que existen recursos suficientes para almacenar los datos generados.

2. Herramientas de planificación

Las tareas administrativas pueden planificarse desde diferentes niveles del sistema:

2.1. Planificador integrado en MariaDB

MariaDB dispone de un planificador de eventos que permite ejecutar instrucciones SQL automáticamente en fechas o intervalos determinados.

Los eventos se almacenan en el propio servidor y se ejecutan cuando se cumple la planificación definida.

Es adecuado para automatizar operaciones como:

- Eliminar registros antiguos.
- Actualizar datos de mantenimiento.
- Generar resúmenes estadísticos.
- Ejecutar procedimientos almacenados.
- Actualizar información de tablas relacionadas.

2.2. Planificador del sistema operativo

Debian dispone de herramientas externas para programar tareas. Una de las más utilizadas es `cron`.

A diferencia de los eventos de MariaDB, `cron` permite ejecutar comandos del sistema operativo. Por ejemplo:

- Ejecutar archivos SQL mediante el cliente de MariaDB.
- Generar copias de seguridad con `mariadb-dump`.
- Comprimir y organizar archivos.
- Copiar copias de seguridad a otros directorios o servidores.
- Ejecutar scripts de mantenimiento escritos en Bash.

2.3. Comparación

Característica	Eventos MariaDB	cron (Debian)
Ámbito	Servidor de bases de datos.	Sistema operativo.
Ejecuta SQL	Directamente.	Mediante el cliente MariaDB.
Ejecuta comandos del sistema	No directamente.	Sí.
Copias con mariadb-dump	No es su función habitual.	Sí.
Dependencia principal	Planificador de eventos activo en MariaDB.	Servicio cron activo en Debian.

3. Eventos programados de MariaDB

Un evento programado es un objeto de la base de datos que permite ejecutar una o varias instrucciones SQL automáticamente en un momento determinado.

Para crear eventos se utiliza la instrucción `CREATE EVENT`.

3.1. Comprobar el planificador de eventos

Antes de crear eventos debemos comprobar si el planificador está activado.

```
SHOW VARIABLES LIKE 'event_scheduler';
```

El resultado puede mostrar alguno de estos valores:

Valor	Significado
ON	El planificador está activado.
OFF	El planificador está desactivado.
DISABLED	El planificador no está disponible porque se ha deshabilitado en la configuración del servidor.

3.2. Activar el planificador

Si el planificador está desactivado, un usuario con los privilegios necesarios puede activarlo mediante:

```
SET GLOBAL event_scheduler = ON;
```

Esta modificación afecta al funcionamiento actual del servidor. Para comprobar el resultado:

```
SHOW VARIABLES LIKE 'event_scheduler';
```

> **Persistencia de la configuración**
> La instrucción `SET GLOBAL` modifica el valor en ejecución, pero no garantiza que se conserve después de reiniciar MariaDB.
>
> Para mantenerlo activado, hay que configurar el servidor en Debian.

3.3. Configuración permanente en Debian

En Debian, la configuración de MariaDB suele organizarse mediante archivos situados en:

```
/etc/mysql/mariadb.conf.d/
```

Podemos crear un archivo específico para la configuración del planificador:

```
sudo nano /etc/mysql/mariadb.conf.d/60-event-scheduler.cnf
```

Introducimos el siguiente contenido:

```
[mariadbd]
event_scheduler = ON
```

Guardamos el archivo y reiniciamos el servicio:

```
sudo systemctl restart mariadb
```

Comprobamos el estado:

```
sudo systemctl status mariadb
```

Y verificamos desde MariaDB:

```
SHOW VARIABLES LIKE 'event_scheduler';
```

4. Creación de eventos mediante CREATE EVENT

4.1. Sintaxis general

```
CREATE EVENT nombre_evento
ON SCHEDULE planificación
DO
    instrucción_sql;
```

Los elementos principales son:

- `CREATE EVENT`: crea el evento programado.
- `nombre_evento`: identifica el evento.
- `ON SCHEDULE`: establece cuándo debe ejecutarse.
- `DO`: indica las instrucciones que se ejecutarán.

4.2. Evento de ejecución única: AT

La cláusula `AT` permite programar una tarea para una fecha y hora determinadas.

```
CREATE EVENT ev_limpieza_unica
ON SCHEDULE AT '2026-12-01 02:00:00'
DO
    DELETE FROM registros_temporales
    WHERE fecha < CURRENT_DATE - INTERVAL 30 DAY;
```

Este evento se ejecutará una sola vez en la fecha y hora indicadas, siempre que el planificador esté activo y el evento esté habilitado.

4.3. Evento periódico: EVERY

La cláusula `EVERY` permite establecer un intervalo de repetición.

Por ejemplo, para ejecutar una tarea cada día:

```
CREATE EVENT ev_limpieza_diaria
ON SCHEDULE EVERY 1 DAY
DO
    DELETE FROM registros_temporales
    WHERE fecha < CURRENT_DATE - INTERVAL 30 DAY;
```

También podemos utilizar otros intervalos:

```
-- Cada hora
ON SCHEDULE EVERY 1 HOUR

-- Cada 15 minutos
ON SCHEDULE EVERY 15 MINUTE

-- Cada semana
ON SCHEDULE EVERY 1 WEEK

-- Cada mes
ON SCHEDULE EVERY 1 MONTH
```

4.4. Definir el inicio y el final de un evento

Podemos establecer cuándo comienza la planificación y cuándo debe dejar de repetirse mediante `STARTS` y `ENDS`.

```
CREATE EVENT ev_informe_periodico
ON SCHEDULE EVERY 1 DAY
STARTS CURRENT_TIMESTAMP + INTERVAL 1 DAY
ENDS CURRENT_TIMESTAMP + INTERVAL 30 DAY
DO
    INSERT INTO informes (fecha_generacion, descripcion)
    VALUES (NOW(), 'Informe diario generado');
```

En este ejemplo, el evento se ejecutará diariamente durante el periodo comprendido entre las fechas de inicio y finalización.

4.5. Crear un evento con varias instrucciones

Cuando necesitamos ejecutar varias instrucciones, podemos utilizar un bloque `BEGIN ... END`.

```
DELIMITER //

CREATE EVENT ev_actualizar_resumen
ON SCHEDULE EVERY 1 DAY
DO
BEGIN
    UPDATE productos
    SET stock = 0
    WHERE stock < 0;

    INSERT INTO registro_tareas (
        tarea,
        fecha_ejecucion
    )
    VALUES (
        'Actualización de productos',
        NOW()
    );
END//

DELIMITER ;
```

El ejemplo presupone que existe la tabla `registro_tareas`. El evento ejecuta ambas instrucciones en cada activación.

5. Gestión de eventos programados

5.1. Consultar los eventos existentes

```
SHOW EVENTS;
```

También podemos consultar la definición de un evento concreto:

```
SHOW CREATE EVENT ev_limpieza_diaria;
```

5.2. Desactivar temporalmente un evento

Si queremos impedir que un evento se ejecute sin eliminarlo, podemos desactivarlo:

```
ALTER EVENT ev_limpieza_diaria DISABLE;
```

5.3. Activar un evento

```
ALTER EVENT ev_limpieza_diaria ENABLE;
```

5.4. Eliminar un evento

```
DROP EVENT IF EXISTS ev_limpieza_diaria;
```

5.5. Consultar la tabla de eventos

Los eventos se almacenan en el esquema del sistema `mysql`. También pueden consultarse mediante:

```
SELECT
    EVENT_NAME,
    EVENT_TYPE,
    STATUS,
    INTERVAL_VALUE,
    INTERVAL_FIELD,
    STARTS,
    ENDS
FROM information_schema.EVENTS
WHERE EVENT_SCHEMA = 'tienda';
```

6. Ejemplo completo: limpieza automática de registros

Vamos a crear una tabla que almacene registros temporales y un evento que elimine automáticamente aquellos que tengan más de 30 días.

6.1. Crear la tabla

```
CREATE DATABASE IF NOT EXISTS mantenimiento;
USE mantenimiento;

CREATE TABLE registros_temporales (
    id INT AUTO_INCREMENT PRIMARY KEY,
    descripcion VARCHAR(200),
    fecha DATETIME NOT NULL
);
```

6.2. Insertar datos de prueba

```
INSERT INTO registros_temporales (descripcion, fecha)
VALUES
('Registro antiguo 1', NOW() - INTERVAL 45 DAY),
('Registro antiguo 2', NOW() - INTERVAL 60 DAY),
('Registro reciente 1', NOW() - INTERVAL 5 DAY),
('Registro reciente 2', NOW());
```

Comprobamos los registros existentes:

```
SELECT *
FROM registros_temporales;
```

6.3. Crear un evento periódico

```
DELIMITER //

CREATE EVENT ev_limpieza_registros
ON SCHEDULE EVERY 1 DAY
DO
BEGIN
    DELETE FROM registros_temporales
    WHERE fecha < NOW() - INTERVAL 30 DAY;
END//

DELIMITER ;
```

6.4. Comprobar el evento

```
SHOW EVENTS FROM mantenimiento;
```

El evento está programado para ejecutarse diariamente. Para comprobar inmediatamente la lógica de limpieza, podemos ejecutar manualmente la misma instrucción:

```
DELETE FROM registros_temporales
WHERE fecha < NOW() - INTERVAL 30 DAY;

SELECT *
FROM registros_temporales;
```

> **Importante**
> Ejecutar manualmente la instrucción permite comprobar su resultado, pero no equivale a comprobar que el planificador ejecuta el evento. Para verificar la planificación hay que consultar su estado y comprobar su ejecución cuando llegue el momento programado.

7. Automatización mediante cron en Debian

Los eventos de MariaDB son útiles para ejecutar instrucciones SQL dentro del servidor. Sin embargo, algunas tareas requieren ejecutar comandos del sistema operativo.

Para estas operaciones podemos utilizar **cron**, un servicio de Debian que permite programar comandos para que se ejecuten automáticamente en fechas y horas determinadas.

7.1. Comprobar el servicio cron

Comprobamos si está instalado:

```
dpkg -l cron
```

Si no está instalado:

```
sudo apt update
sudo apt install cron
```

Comprobamos su estado:

```
sudo systemctl status cron
```

Si fuese necesario, lo activamos:

```
sudo systemctl enable --now cron
```

7.2. Programar tareas con crontab

Cada usuario puede disponer de su propia lista de tareas programadas. Para editarla utilizamos:

```
crontab -e
```

Para consultar las tareas programadas:

```
crontab -l
```

Las tareas se definen mediante una expresión formada por cinco campos temporales y el comando que se desea ejecutar.

7.3. Estructura de una tarea cron

```
MINUTO HORA DIA_MES MES DIA_SEMANA COMANDO
```

Campo	Valores habituales	Significado
Minuto	0-59	Minuto de ejecución.
Hora	0-23	Hora de ejecución.
Día del mes	1-31	Día del mes.
Mes	1-12	Mes del año.
Día de la semana	0-7	Día de la semana. El domingo puede representarse con 0 o 7.

Algunos operadores importantes:

- `*`: cualquier valor del campo.
- `,`: permite indicar varios valores.
- `-`: indica un intervalo de valores.
- `/`: establece intervalos de repetición.

7.4. Ejemplos de planificación

Expresión	Significado
`0 2 * * *`	Todos los días a las 02:00.
`30 3 * * 1`	Todos los lunes a las 03:30.
`0 1 1 * *`	El día 1 de cada mes a la 01:00.
`*/15 * * * *`	Cada 15 minutos.
`0 8-18 * * 1-5`	Cada hora en punto, de 8:00 a 18:00, de lunes a viernes.

8. Ejecutar scripts SQL desde cron

Para ejecutar un archivo SQL de forma automática podemos utilizar el cliente de MariaDB desde la terminal de Debian.

8.1. Crear un archivo SQL

Vamos a crear un archivo que contenga instrucciones para realizar una tarea administrativa.

```
mkdir -p ~/scripts
nano ~/scripts/mantenimiento.sql
```

Introducimos el siguiente contenido:

```
USE mantenimiento;

DELETE FROM registros_temporales
WHERE fecha < NOW() - INTERVAL 30 DAY;

INSERT INTO registro_tareas (tarea, fecha_ejecucion)
VALUES ('Limpieza automática', NOW());
```

El ejemplo presupone que existe la tabla `registro_tareas`. Podemos crearla previamente:

```
CREATE TABLE registro_tareas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tarea VARCHAR(150) NOT NULL,
    fecha_ejecucion DATETIME NOT NULL
);
```

8.2. Ejecutar el script manualmente

Antes de programar su ejecución, debemos comprobar que funciona correctamente.

```
mariadb -u usuario_admin -p < ~/scripts/mantenimiento.sql
```

El parámetro `-u` indica el usuario de MariaDB y `-p` solicita su contraseña.

También podemos ejecutar un archivo SQL desde una sesión interactiva:

```
SOURCE /home/usuario/scripts/mantenimiento.sql;
```

8.3. Programar su ejecución

Editamos el crontab del usuario que ejecutará la tarea:

```
crontab -e
```

Añadimos una línea para ejecutar el script todos los días a las 02:00:

```
0 2 * * * /usr/bin/mariadb -u usuario_admin -p'CLAVE' < /home/usuario/scripts/mantenimiento.sql
```

> **Seguridad de las credenciales**
> No es recomendable escribir contraseñas directamente en el crontab, porque pueden quedar expuestas al consultar la configuración del usuario.
>
> Una alternativa es utilizar un archivo de configuración protegido, por ejemplo `~/.my.cnf`, con permisos restrictivos.

8.4. Utilizar un archivo de configuración de MariaDB

Podemos crear un archivo de configuración para que el cliente MariaDB obtenga las credenciales sin tener que incluirlas directamente en el comando.

```
nano ~/.my.cnf
```

Contenido:

```
[client]
user=usuario_admin
password=CLAVE
host=localhost
```

Protegemos el archivo para que solo el propietario pueda leerlo:

```
chmod 600 ~/.my.cnf
```

Ahora podemos ejecutar el script sin incluir las credenciales:

```
mariadb < /home/usuario/scripts/mantenimiento.sql
```

La entrada de cron quedaría de la siguiente manera:

```
0 2 * * * /usr/bin/mariadb < /home/usuario/scripts/mantenimiento.sql
```

El usuario de Debian que ejecuta cron debe tener acceso al archivo de configuración y utilizar una cuenta de MariaDB con los privilegios mínimos necesarios.

9. Copias de seguridad automatizadas con mariadb-dump

Una de las tareas administrativas más importantes es la realización periódica de copias de seguridad.

MariaDB proporciona la herramienta `mariadb-dump`, que permite exportar la estructura y los datos de una base de datos a un archivo SQL.

9.1. Realizar una copia manual

Para realizar una copia de la base de datos `tienda`:

```
mariadb-dump -u usuario_admin -p tienda > tienda.sql
```

El archivo `tienda.sql` contendrá instrucciones SQL para reconstruir la base de datos y sus datos.

9.2. Copiar todas las bases de datos

```
mariadb-dump -u usuario_admin -p --all-databases > todas_las_bases.sql
```

Esta opción exporta todas las bases de datos a un único archivo. El usuario debe disponer de los privilegios necesarios para acceder a la información que se desea exportar.

9.3. Crear un directorio para las copias

```
mkdir -p ~/copias_mariadb
```

Conviene utilizar nombres de archivo que permitan identificar la fecha de realización de cada copia.

9.4. Copia con fecha y hora

```
mariadb-dump -u usuario_admin tienda \
> ~/copias_mariadb/tienda_$(date +%Y%m%d_%H%M%S).sql
```

La expresión `$(date ...)` ejecuta el comando `date` y utiliza su resultado para construir el nombre del archivo.

Por ejemplo, una copia realizada el 12 de noviembre de 2026 a las 02:00 podría llamarse:

```
tienda_20261112_020000.sql
```

9.5. Comprimir las copias

Para reducir el espacio ocupado podemos comprimir la salida directamente:

```
mariadb-dump -u usuario_admin tienda | gzip \
> ~/copias_mariadb/tienda_$(date +%Y%m%d_%H%M%S).sql.gz
```

9.6. Programar una copia diaria con cron

Editamos el crontab:

```
crontab -e
```

Añadimos la siguiente tarea para realizar una copia todos los días a las 02:30:

```
30 2 * * * /usr/bin/mariadb-dump tienda | gzip > /home/usuario/copias_mariadb/tienda_$(date +\%Y\%m\%d_\%
```

En las entradas de crontab, el carácter `%` tiene un significado especial. Por eso debe escaparse con una barra invertida cuando se utiliza dentro de un comando.

> **Recomendación**
> Una copia de seguridad solo es útil si puede recuperarse. Es necesario comprobar periódicamente los archivos generados y realizar pruebas de restauración en una base de datos independiente.

10. Registrar resultados y gestionar errores

Una tarea automatizada debe permitir conocer si se ha ejecutado, cuándo lo ha hecho y si ha terminado correctamente.

Si una tarea falla y no existe ningún mecanismo de registro, puede pasar desapercibida durante mucho tiempo.

10.1. Redirigir la salida a un archivo de registro

En Linux podemos utilizar los operadores de redirección para guardar la salida de los comandos.

- `>` : crea o sobrescribe un archivo.
- `>>` : añade información al final de un archivo.
- `2>` : redirige los mensajes de error.
- `2>&1` : combina la salida de error con la salida estándar.

Ejemplo de registro de una ejecución:

```
mariadb-dump tienda > copia.sql \
2>> /home/usuario/logs/copias_error.log
```

10.2. Crear un script Bash para las copias

En lugar de introducir toda la lógica directamente en cron, podemos crear un script Bash que realice la copia y registre el resultado.

Creamos el directorio de scripts y registros:

```
mkdir -p ~/scripts
mkdir -p ~/logs
mkdir -p ~/copias_mariadb
```

Creamos el archivo:

```
nano ~/scripts/copia_tienda.sh
```

Contenido del script:

```
#!/bin/bash

# Configuración
BASE_DATOS="tienda"
DIRECTORIO="/home/usuario/copias_mariadb"
LOG="/home/usuario/logs/copias.log"

# Fecha para identificar la copia
FECHA=$(date +%Y%m%d_%H%M%S)

# Archivo de destino
ARCHIVO="$DIRECTORIO/${BASE_DATOS}_${FECHA}.sql.gz"

# Crear directorios si no existen
mkdir -p "$DIRECTORIO"
mkdir -p "$(dirname "$LOG")"

# Realizar la copia
if mariadb-dump "$BASE_DATOS" | gzip > "$ARCHIVO"; then

    echo "[$(date)] Copia correcta: $ARCHIVO" >> "$LOG"

else

    echo "[$(date)] ERROR al realizar la copia" >> "$LOG"
    rm -f "$ARCHIVO"
    exit 1

fi
```

El script utiliza el archivo `~/.my.cnf` para obtener las credenciales de conexión.

10.3. Dar permisos de ejecución

```
chmod +x ~/scripts/copia_tienda.sh
```

10.4. Ejecutar manualmente

```
~/scripts/copia_tienda.sh
```

Comprobamos los archivos generados:

```
ls -lh ~/copias_mariadb/
```

Consultamos el registro:

```
cat ~/logs/copias.log
```

10.5. Programar el script con cron

Editamos el crontab del usuario:

```
crontab -e
```

Añadimos:

```
30 2 * * * /home/usuario/scripts/copia_tienda.sh
```

De esta forma, el script se ejecutará todos los días a las 02:30.

10.6. Comprobar los registros de cron

Debian utiliza el sistema de registros del sistema para almacenar información sobre la ejecución de servicios y tareas.

Podemos consultar los mensajes del servicio cron mediante:

```
sudo journalctl -u cron
```

Para consultar los últimos mensajes:

```
sudo journalctl -u cron -n 50
```

En instalaciones que utilizan registros tradicionales también puede consultarse:

```
sudo grep CRON /var/log/syslog
```

Atención a los errores de los scripts
En el ejemplo anterior, el código de salida de una tubería Bash puede corresponder únicamente al último comando ejecutado. Para detectar también errores de `mariadb-dump`, conviene utilizar `set -o pipefail` al principio del script.

Por tanto, una versión más robusta debe comenzar con:

```
#!/bin/bash
set -o pipefail
```

11. Restauración de una copia de seguridad

La automatización de copias debe complementarse con procedimientos de restauración. No basta con generar archivos: debemos asegurarnos de que contienen información recuperable.

11.1. Restaurar una copia SQL

Si la copia se ha generado en formato SQL sin comprimir:

```
mariadb -u usuario_admin tienda < tienda.sql
```

11.2. Restaurar una copia comprimida

Si se ha utilizado `gzip`:

```
gunzip -c tienda.sql.gz | mariadb -u usuario_admin tienda
```

El usuario de MariaDB debe tener los permisos necesarios para crear o modificar los objetos incluidos en la copia.

11.3. Buenas prácticas para las copias

- Guardar las copias en un directorio distinto al de los datos originales.
- Utilizar nombres que incluyan fecha y hora.
- Proteger los archivos frente a accesos no autorizados.
- Conservar varias versiones de las copias.
- Eliminar las copias antiguas siguiendo una política de retención.
- Guardar copias en otro dispositivo o ubicación cuando sea posible.
- Realizar pruebas periódicas de restauración.
- Registrar las ejecuciones y comprobar los errores.

12. Comparación: eventos MariaDB y cron

Ambas herramientas permiten automatizar tareas, pero tienen finalidades diferentes y pueden utilizarse conjuntamente.

Situación	Herramienta recomendada
Eliminar registros antiguos de una tabla cada día.	Evento MariaDB.
Actualizar automáticamente el estado de registros.	Evento MariaDB.
Ejecutar un procedimiento almacenado cada semana.	Evento MariaDB.
Generar una copia con mariadb-dump.	cron.
Comprimir archivos y eliminar copias antiguas.	cron.
Enviar una copia a otro servidor.	cron y herramientas del sistema.
Ejecutar un script Bash de mantenimiento.	cron.

13. Resumen del bloque

Concepto	Descripción
Planificación	Programar la ejecución de tareas administrativas.
event_scheduler	Variable que controla el planificador de eventos de MariaDB.
CREATE EVENT	Crea un evento programado.
ON SCHEDULE AT	Programa una ejecución única.
ON SCHEDULE EVERY	Programa ejecuciones periódicas.
ALTER EVENT	Modifica o habilita/deshabilita un evento.
DROP EVENT	Elimina un evento.
cron	Planificador de tareas del sistema operativo.
crontab	Permite consultar y editar las tareas programadas.
mariadb-dump	Genera copias de seguridad en formato SQL.
Redirecciones	Permiten guardar salidas y errores en archivos.
journalctl	Permite consultar los registros de servicios del sistema.

14. Ejercicios prácticos de comprobación

Ejercicio 1. Planificador de eventos

Comprueba el estado de `event_scheduler` en el servidor MariaDB. Actívalo si está deshabilitado y verifica que el cambio se ha aplicado.

Ejercicio 2. Evento de mantenimiento

Crea una tabla para almacenar registros temporales y programa un evento que elimine diariamente aquellos que tengan más de 30 días. Comprueba la definición y el estado del evento.

Ejercicio 3. Gestión de eventos

Crea un evento periódico, desactívalo, vuelve a activarlo y consulta su definición. Finalmente, elimínalo.

Ejercicio 4. Programación con cron

Crea un archivo SQL que realice una operación de mantenimiento sobre una base de datos. Ejecútalo manualmente desde Debian y programa su ejecución periódica mediante `cron`.

Ejercicio 5. Copia de seguridad automatizada

Genera una copia de seguridad de una base de datos utilizando `mariadb-dump`. Incluye la fecha y hora en el nombre del archivo y comprueba que se ha generado correctamente.

Ejercicio 6. Registro de resultados

Crea un script Bash que realice una copia de seguridad, registre la fecha de ejecución y permita identificar si la operación ha terminado correctamente o ha producido un error.

Ejercicio 7. Restauración

Utiliza una copia de seguridad generada anteriormente para restaurar la información en una base de datos de prueba. Comprueba que se han recuperado correctamente la estructura y los datos.

> **Conclusión**
> La planificación de tareas permite automatizar operaciones administrativas y mejorar la disponibilidad y el mantenimiento de un sistema gestor de bases de datos. Los eventos de MariaDB son apropiados para ejecutar instrucciones SQL dentro del servidor, mientras que `cron` permite integrar las tareas de administración de la base de datos con las herramientas del sistema operativo.
>
> Una automatización fiable debe incluir controles, registros, gestión de errores y comprobaciones periódicas de los resultados.